RETAIL DATA AGENT · QUERY GATEWAY · AS OF 2026-11-01

How does the parser turn SQL into a tree, and how do our rules catch a bad query?

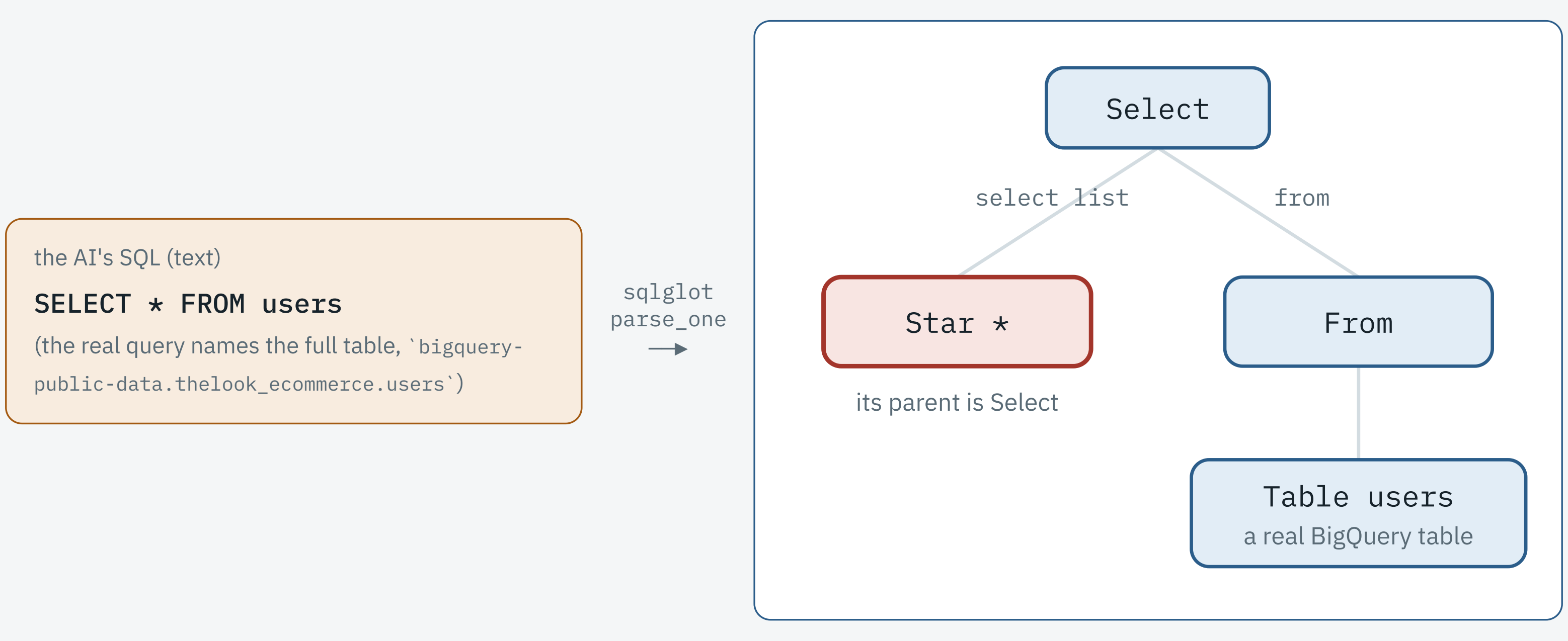
Before any SQL runs, our code turns the AI's text into a tree of labelled pieces, then asks that tree a few yes/no questions in order. The first "no" stops the query, and BigQuery never sees it. See also one query's full journey, brand scoping and what happens when a query fails.

☐ SQL text the AI wrote ☐ parse tree ☐ check passed ☐ check stopped it ☐ our code rewrites

In short

1. A library called **sqlglot** reads the SQL text and builds a tree: one labelled box (a *node*) per piece, such as `Select`, `Star` (⋆), `Column` and `Table`.
2. Our checks walk that tree and ask questions such as "is every table one of the 4?" or "is there a ⋆ that isn't inside `COUNT`?".
3. A "no" calls `raise`: Python stops right there, the AI gets an `Error: …` message, and BigQuery is never called.
4. The checks only *read* the tree. The text that goes to BigQuery is the AI's own text, rewritten in up to three places: a total of age, city and the like gets a `HAVING` that hides groups under 5 customers, customer ids become pseudonyms (for everyone), and tables are filtered to the user's brands (for brand managers).

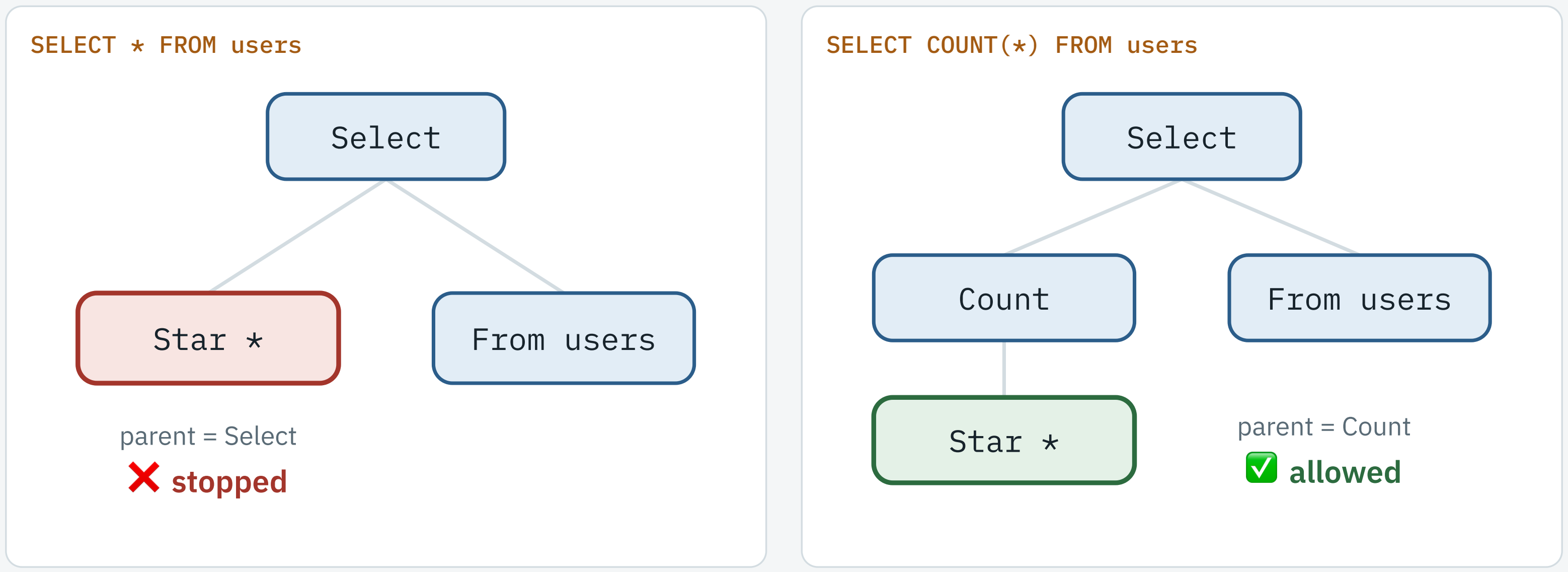
Step 1: the parser turns text into a tree



The parser doesn't look anything up and doesn't change anything. It only labels the pieces. The ⋆ becomes a node called `Star`. Nobody has worked out yet which columns it stands for, and nobody needs to: **the fact that a `Star` sits directly under `Select` is enough to know the query asks for every column**, including email.

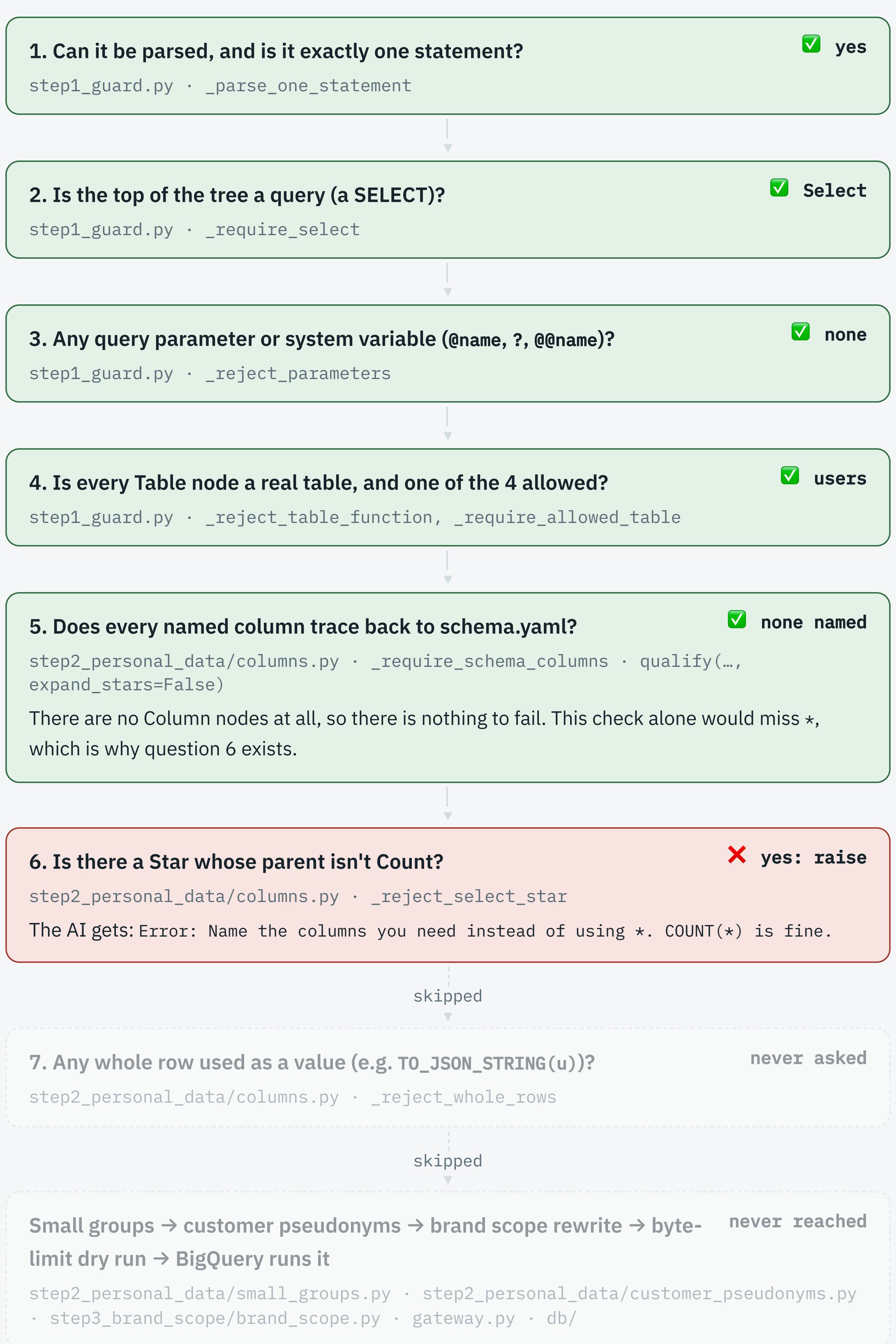
Same symbol, different parent

The ⋆ in `COUNT(⋆)` becomes the same `Star` node. The only difference is the box directly above it, its *parent*. That's how the rule tells "count the rows" (fine) from "give me every column" (stopped).



Step 2: the questions, one after another

The gateway runs the checks in this order, all on the same text. Each check builds its own tree from it. Here is `SELECT ⋆ FROM users` going through:



1. Can it be parsed, and is it exactly one statement? ✅ yes
`step1_guard.py · _parse_one_statement`

2. Is the top of the tree a query (a SELECT)? ✅ Select
`step1_guard.py · _require_select`

3. Any query parameter or system variable (`@name`, `?`, `@@name`)? ✅ none
`step1_guard.py · _reject_parameters`

4. Is every Table node a real table, and one of the 4 allowed? ✅ users
`step1_guard.py · _reject_table_function, _require_allowed_table`

5. Does every named column trace back to schema.yaml? ✅ none named
`step2_personal_data/columns.py · _require_schema_columns · qualify(…, expand_stars=False)`
There are no Column nodes at all, so there is nothing to fail. This check alone would miss ⋆, which is why question 6 exists.

6. Is there a Star whose parent isn't Count? ✗ yes: raise
`step2_personal_data/columns.py · _reject_select_star`
The AI gets: `Error: Name the columns you need instead of using ⋆. COUNT(⋆) is fine.`

skipped

7. Any whole row used as a value (e.g. `TO_JSON_STRING(u)`)? never asked
`step2_personal_data/columns.py · _reject_whole_rows`

skipped

Small groups → customer pseudonyms → brand scope rewrite → byte-limit dry run → BigQuery runs it never reached
`step2_personal_data/small_groups.py · step2_personal_data/customer_pseudonyms.py · step3_brand_scope/brand_scope.py · gateway.py · db/`

The AI reads the error and names the columns instead. In the live run on 2026-10-31 it sent `SELECT id, age, gender, city, state, country, traffic_source, created_at FROM users LIMIT 3`, which passed these seven questions. Since 2026-11-01 it is stopped one step later: age, gender, city, state, country and traffic source may appear only inside totals (rule 10 below, and small groups). `SELECT id, created_at FROM users LIMIT 3` passes everything.

The code that stopped it

```
# step2_personal_data/columns.py: check() reads like a checklist, one named rule per line
def check(self, sql):
    statement = sqlglot.parse_one(sql, read=self._dialect)   # text → tree
    self._require_schema_columns(statement)                  # Q5: trace each column; keep ⋆ as written
    _reject_select_star(statement)                           # Q6: stops SELECT ⋆ here
    _reject_whole_rows(statement)                            # Q7: a table used as a value

def _reject_select_star(statement):
    for star in statement.find_all(exp.Star):                # every ⋆ anywhere in the tree
        if not isinstance(star.parent, exp.Count):           # the box directly above it
            raise QueryRejected("Name the columns you need …")  # stop here
```

`raise` means "stop this function now and jump out with an error". In `gateway.run`, every line after the check (customer pseudonyms, brand scope, byte check, `database.execute`) is skipped, and the message goes back to the AI as `Error: …`.

The trap: why `expand_stars=False`

`qualify` can also replace ⋆ in the tree with the columns it knows from schema.yaml. That only changes the tree, not the text BigQuery runs.

Expansion on (the library default)

- The tree becomes `SELECT id, age, city, …`. No Star is left, so question 6 finds nothing.
- The check passes ✅, but BigQuery still runs the text `SELECT ⋆` and returns email ❌.

Expansion off (what we do)

- The tree keeps its `Star`, exactly as the AI wrote it.
- Question 6 sees it and stops the query before BigQuery.

Every rule that uses the parse tree

Each row is one question asked of the tree, in the order the gateway asks them. The first "no" wins.

#	QUESTION ASKED OF THE TREE	CAUGHT EXAMPLE	WHAT THE AI IS TOLD
`step1_guard.py · which statement, which tables`			
1	Can it be parsed at all? If not, we can't check it, so it doesn't run.	`SELECT FROM WHERE`	The query could not be parsed, so it was not run. Simplify it.
2	Exactly one statement?	`SELECT 1 …; DROP TABLE …`	Send one statement per query.
3	Is the top node a query (SELECT or UNION of SELECTs)?	`DELETE FROM users WHERE TRUE`	Only SELECT queries are allowed.
4	Any query parameter or system variable? Our own SQL sends the pseudonym salt as `@pseudonym_salt`, so the AI must not be able to read one.	`SELECT @pseudonym_salt` `SELECT @@project_id`	Query parameters and system variables (@name, ?, @@name) are not allowed. Write values into the SQL.
5	Is every Table node a plain name, not a function?	`SELECT ⋆ FROM ML.PREDICT(…)`	Read only from tables. Table functions such as ML.PREDICT are not allowed.
6	Is every Table one of the 4 allowed? (A one-part name the query defines with `WITH` is skipped.)	`SELECT ip_address FROM events`	Table `…events` is not allowed. Use only these tables: …
`step2_personal_data/columns.py · which columns (personal data)`			
7	Does every column trace back, through aliases, joins, subqueries and WITH, to a column in schema.yaml? Is it clear which table it's from?	`SELECT email FROM users` `SELECT gender FROM users u JOIN orders o …`	Read only the columns listed in the schema. When a column is in two joined tables, prefix it with the table alias. (Column 'email' could not be resolved.)

8	Is there a Star whose parent isn't Count?	`SELECT ⋆ FROM users`
		`SELECT u.⋆ …`
		`SELECT ⋆ EXCEPT (id) …`
			Name the columns you need instead of using *. COUNT(*) is fine.

9	Is a whole table row used as a value (a TableColumn node)?	`SELECT u FROM users u` `TO_JSON_STRING(u)`	Select columns, not whole rows: `u` is a table, not a column.
`step2_personal_data/small_groups.py · demographic columns only in totals of 5+ customers`			

10	For every SELECT that reads a column tagged `aggregate_only` in schema.yaml (age, gender, city, state, country, traffic source) straight from a table: is it a total, meaning a GROUP BY or an aggregate that isn't inside a window (`OVER`)? A WITH step that only passes the column along counts as a listing.	`SELECT id, city FROM users`
		`SELECT DISTINCT country …`
		`COUNT(⋆) OVER (PARTITION BY city)`
			`users.city` can be used only inside a total (COUNT, SUM, AVG … with GROUP BY), in the same SELECT that reads the table …

11	Does that total use ARRAY_AGG, STRING_AGG or ARRAY_CONCAT_AGG? They give one row per group, but it holds every customer's value. If not, add `HAVING COUNT(DISTINCT <customer id>) >= 5` to that same SELECT (ANDed with the AI's own HAVING), so BigQuery drops small groups before LIMIT.	`SELECT ARRAY_AGG(city) FROM users` `GROUP BY city → GROUP BY city HAVING COUNT(DISTINCT users.id) >= 5`	`users.city` can't be collected with ARRAY_AGG or STRING_AGG … For a plain total: nothing, the HAVING is added quietly.
`step2_personal_data/customer_pseudonyms.py · rewrites the tree (every user, the CEO too)`			
12	Swap each customer table the query reads (`users`, `orders`, `order_items`) for a copy whose customer id is a salted hash, and put those copies in the WITH clause at the front. BigQuery builds the copies while the query runs, so the AI's SQL only ever sees pseudonyms.	`FROM users → FROM _pseudonymized_users AS users`	Nothing; the AI isn't told. It sees ids such as `cust_72ac84091163`, and a follow-up can filter on one.
`step3_brand_scope/brand_scope.py · rewrites the tree (brand managers only; a user with every brand skips it)`			
13	Swap every Table node for a filtered copy (`_brand_scoped_users` …) and put those copies in the WITH clause at the front. The tree is turned back into text, and **that** text runs.	`FROM users → FROM _brand_scoped_users AS users`	Nothing; the AI isn't told. A table with no brand filter is stopped: "can't be limited to your brands".

Not parser rules: the byte limit (a BigQuery dry run of the final text, in query_safety/gateway.py) and the row cap (the database adapter). They run after all of the above.

> **The rule in one sentence**
>
> The parser only labels the pieces of the SQL. Our checks look at those labels and their parents, and the first "no" raises an error before BigQuery is ever called.

Code: `QueryGuard.check` in `src/retail_agent/query_safety/step1_guard.py`; `PersonalDataCheck.check` in `src/retail_agent/query_safety/step2_personal_data/columns.py`; `SmallGroups.apply` in `src/retail_agent/query_safety/step2_personal_data/small_groups.py`; `CustomerPseudonyms.apply` in `src/retail_agent/query_safety/step2_personal_data/customer_pseudonyms.py`; `BrandScope.apply` in `src/retail_agent/query_safety/step3_brand_scope/brand_scope.py`; called in that order by `QueryGateway.run` in `src/retail_agent/query_safety/gateway.py`, wired in `assemble` (`src/retail_agent/app.py`).

Library: sqlglot 30.27 (`parse_one`, `find_all`, `optimizer.qualify.qualify`), BigQuery dialect.

Tests: `tests/test_step1_guard.py`, `tests/test_step2_personal_data.py`, `tests/test_step2_small_groups.py`, `tests/test_step2_customer_pseudonyms.py`, `tests/test_step3_brand_scope.py`. Rejections are logged as `SQL rejected: …` in `logs/retail-agent.log`.

Tickets: `tasks/08-block-personal-data-columns.md`, `tasks/09-hash-customer-ids.md`, `tasks/10-demographics-only-in-totals-of-5.md`. Plan: step 2 and step 3 in `docs/plan.md`. See also one query's full journey, brand scoping, architecture, SQL errors, small groups.